Number Pattern Generator

Java Console Application

Cognifyz Technologies — Java Programming Internship
Task 2: Number Pattern Generator
Internship Submission Report

Project Title	Number Pattern Generator
Internship	Cognifyz Technologies - Java Programming Internship
Task Number	Task 2
Language	Java (JDK 8+)
Tools Used	VS Code, Java Extension Pack, javac / java
Submission Type	Console Application

1. Objective

The objective of this task is to design and implement a Java console application that generates a variety of classic number patterns based on user-selected options. The program is required to present an interactive text menu, accept the desired pattern type and the number of rows from the user, validate all input, and display the resulting pattern using only loop constructs and user-defined methods.

2. Problem Statement

Build a menu-driven Java program that can generate four distinct number patterns: a Half Pyramid, an Inverted Pyramid, Floyd's Triangle, and a Number Square. The user must be able to select a pattern from a menu, provide the number of rows, and view the pattern printed to the console. The application must handle invalid input gracefully (such as non-numeric input, out-of-range menu choices, and invalid row counts) and must allow the user to exit cleanly through a dedicated menu option.

3. Tools & Technologies Used

- Programming Language: Java (JDK 8 or above)
- IDE: Visual Studio Code with the Extension Pack for Java
- Compiler / Runtime: javac and java (standard JDK command-line tools)
- I/O: java.util.Scanner for reading console input
- Version Control friendly structure (plain .java source, no build tool required)

4. Project Structure

```
PatternGenerator/
■■■ src/
■   ■■■ PatternGenerator.java   (complete source code)
■■■ .vscode/
■   ■■■ launch.json             (VS Code run configuration)
■   ■■■ settings.json           (VS Code Java project settings)
■■■ Screenshots/                (sample run screenshots)
■■■ README.md                   (setup & usage guide)
■■■ report.pdf                  (this report)
■■■ Output.txt                  (sample console output)
■■■ PatternGenerator.zip        (zipped project deliverable)
```

5. Approach & Methodology

The application is organized around a single public class, **PatternGenerator**, which contains a set of small, single-purpose methods. The **main()** method runs a loop that repeatedly displays the menu and dispatches to the appropriate pattern method using a **switch** statement, until the user selects the Exit option.

5.1 Menu & Input Handling

The **displayMenu()** method prints the five available options. The **readMenuChoice()** method reads the user's selection and validates it inside a **while** loop, re-prompting until a whole number between 1 and 5 is supplied. Similarly, **readRows()** validates the requested row count, ensuring it is a positive whole

number within a sensible range (1 to 100) so the console output remains readable. Both methods rely on a helper, **isPositiveInteger(String)**, which manually inspects each character of the input to confirm it represents a positive integer — avoiding regular expressions to keep the logic transparent and beginner-friendly.

5.2 Pattern Generation Logic

Each pattern is produced by a dedicated method using nested **for** loops only — no arrays, collections, or recursion are used anywhere in the program:

- **printHalfPyramid(rows)** — the outer loop controls the row number (i = 1 to rows); the inner loop prints numbers from 1 up to i.
- **printInvertedPyramid(rows)** — the outer loop counts down from rows to 1; the inner loop prints numbers from 1 up to the current row count, producing an upside-down half pyramid.
- **printFloydsTriangle(rows)** — a running counter variable is incremented on every inner-loop iteration, so numbers continue in sequence across rows instead of restarting at 1.
- **printNumberSquare(rows)** — both loops run from 1 to rows, printing the full sequence 1..rows on every row to form a square grid.

6. Sample Pattern Output (rows = 5)

Half Pyramid	Inverted Pyramid
1 1 2 1 2 3 1 2 3 4 1 2 3 4 5	1 2 3 4 5 1 2 3 4 1 2 3 1 2 1

Floyd's Triangle	Number Square
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5

7. Input Validation

- Menu choice must be a whole number between 1 and 5; any other value (letters, symbols, out-of-range numbers) triggers a re-prompt.
- Row count must be a positive whole number between 1 and 100; zero, negative, non-numeric, or excessively large values are rejected with a clear error message.
- The program never crashes on invalid input — all validation loops keep re-prompting the user instead of throwing exceptions.

8. Testing Summary

Test Case	Input	Expected Result	Status
Half Pyramid, 5 rows	1 → 5	5-row half pyramid printed	Pass
Inverted Pyramid, 5 rows	2 → 5	5-row inverted pyramid printed	Pass
Floyd's Triangle, 5 rows	3 → 5	Sequential numbers 1-15 printed	Pass
Number Square, 5 rows	4 → 5	5x5 grid of repeating 1-5 rows	Pass
Invalid menu input	"abc"	Error message, re-prompt	Pass
Invalid row count	0, 99, "xyz"	Error message, re-prompt	Pass
Exit option	5	Program terminates cleanly	Pass

Full console transcripts of these test runs are available in **Output.txt** in the project root.

9. Conclusion

This task strengthened practical understanding of nested loop control structures, modular method design, and defensive input handling in Java. The resulting application meets all stated requirements: it is menu-driven, generates four distinct number patterns using loops only, validates every user input, and is organized into clear, well-documented methods. The project is fully self-contained and can be compiled and run immediately in Visual Studio Code or from the command line.
